PARTE III — CONSTRUIR PRODUCTOS DIGITALES · CAPÍTULO 10

De la fábrica al código

10

Cómo se construye un producto digital: organizar el trabajo para producir con calidad, adaptarlo cuando todavía no conocemos la solución y comprobar que el resultado sirve a quien lo recibe.

11 nov 2026 · 24 min de lectura · Edición viva 0.10 · Estado: en progreso ·

Una botella puede salir perfecta de la línea y terminar en la basura porque nadie pudo comprarle una tapa. Entre esos dos momentos cabe un negocio entero. También cabe una manera de entender cómo se construye un producto digital.

Antes de hablar de metodologías conviene aclarar qué estamos intentando hacer. Un producto no es sólo un objeto ni una aplicación: es una solución que alguien usa para resolver una necesidad. Fabricarlo implica transformar materiales y coordinar personas; construir su versión digital implica organizar información, decisiones y programas. En ambos casos podemos cumplir todas las instrucciones y, sin embargo, fallarle a quien lo necesita.

A lo largo de este capítulo vamos a recorrer tres preguntas. ¿Cómo se organiza el trabajo para producir con calidad? ¿Cómo se modifica ese modo de trabajar cuando todavía no conocemos la solución? ¿Y cómo sabemos que el resultado sirve una vez que llega al cliente? De cada pregunta surgieron prácticas y metodologías con nombres propios. No hace falta conocerlas de antemano: aparecerán cuando el problema que intentan resolver ya esté sobre la mesa.

Una tapa sobre la mesa





FIGURA 10.1 Seis botellas, seis tapas: lo que el fabricante ve como artículos bien dados de alta, el cliente lo ve como una pieza que quizá no pueda comprar.

Imaginemos una fábrica de botellas térmicas. Al inicio del turno hay cuerpos de acero, tapas, juntas de silicón y cajas de cartón. Una orden indica qué modelo fabricar y cuántas unidades hacen falta. Alguien tiene que convertir esa orden en una secuencia que personas con herramientas distintas puedan ejecutar, comprobar y repetir.

El cuerpo se forma, se suelda y se comprueba su hermeticidad. En otra estación se coloca la junta dentro de la tapa. Las dos piezas se reúnen en ensamble, pasan por una prueba de fuga y reciben una identificación de lote antes del empaque. La tolerancia que el diseñador anotó en un plano tiene ahora una consecuencia bastante íntima para el cliente que lleva la botella junto a su computadora.

Meses después, ese cliente pierde la tapa. Entra al sitio del fabricante y encuentra seis fotografías casi idénticas. La rosca cambió entre dos generaciones, pero en ambas botellas el nombre comercial sigue siendo el mismo. Desde la perspectiva del fabricante hay seis artículos bien dados de alta. Desde la perspectiva del cliente hay una botella que mañana tampoco va a poder usar. Vamos a seguir ese problema desde el taller hasta la pantalla.

Antes de medir la velocidad de la línea, vale la pena distinguir dos actividades que parecen iguales: **hacer trabajo** y **hacer avanzar un producto**. Una persona puede ensamblar tapas sin detenerse y, aun así, no salir ninguna botella porque la siguiente estación está saturada. Cuando hablamos de *flujo* nos referimos a ese recorrido completo, desde que el trabajo entra hasta que entrega algo utilizable. Mirar sólo qué tan ocupada está cada persona puede esconder dónde se detiene el conjunto.

El ritmo de la fábrica



Figura 10.2 — El takt time marca cada cuánto debería salir una botella terminada, no cuánto tarda una persona en fabricarla.

Supongamos que tenemos 420 minutos útiles por turno y pedidos por 210 botellas. Si dividimos el tiempo disponible entre las unidades solicitadas, descubrimos que necesitamos terminar una cada dos minutos. Los sistemas de producción llaman a ese ritmo requerido *takt time*, expresión tomada del alemán *Takt*, que alude al compás. Si la prueba de fuga tarda tres minutos por botella en un único puesto, esa estación puede procesar como máximo 140 durante el turno, aun si trabaja sin interrupción. Acelerar el ensamble sólo hará crecer la fila frente a la prueba.

Este tipo de problemas impulsó una manera distinta de pensar la producción. Durante el siglo XX, Toyota desarrolló un sistema que buscaba fabricar lo necesario, detectar fallas pronto y aprender del trabajo cotidiano sin depender únicamente de grandes inventarios o de inspecciones al final. Décadas después, investigadores y practicantes reunieron varias de esas ideas bajo el nombre **Lean**, que suele traducirse como producción esbelta. Su propósito no es pedir que la gente trabaje más deprisa: es identificar qué pasos producen valor para el cliente y quitar obstáculos que consumen tiempo y recursos sin mejorar el resultado.

Así empezaría una intervención inspirada en Lean. Iría al lugar donde ocurre el trabajo, no sólo a la sala donde se presentan indicadores. En Toyota se usa para ello la palabra japonesa *gemba*:

el sitio real donde podemos comprobar lo que pasa. Acompañaría un lote desde que llega hasta que sale, anotaría por separado el tiempo de transformación y el tiempo de espera, y con la persona que hace la prueba pondría los adaptadores al alcance de la mano. El estándar sirve para comparar dos maneras de trabajar y conservar la que da mejor resultado.

El **Sistema de Producción Toyota**, antecedente central de Lean, se apoya en dos principios complementarios. El primero es **just-in-time**: producir y reponer en el momento y la cantidad que se necesitan, en lugar de llenar la fábrica con piezas por si algún día hacen falta. El segundo es **jidoka**: incorporar la capacidad de reconocer un problema y detener el proceso para no seguir fabricando defectos. Con *just-in-time*, el consumo de tapas en ensamble da la señal para reponerlas; una tarjeta *kanban* —letrero o señal visual, en japonés— puede transmitir esa necesidad. Con *jidoka*, si aparecen juntas mal asentadas, el puesto puede detenerse y pedir ayuda antes de empacar botellas defectuosas.[1]

2 min	140	8
TAKT TIME DEL TURNO (420 min / 210 botellas)	CAPACIDAD MÁXIMA DE LA PRUEBA DE FUGA POR TURNO	PASOS DEL MÉTODO TOYOTA DE RESOLUCIÓN DE PROBLEMAS

Nombrar el desperdicio



Figura 10.3 — Inventario que ocupa un pasillo: una de las siete formas clásicas de muda.

Para saber qué estorba al flujo necesitamos una forma de nombrarlo. En el pensamiento Lean, *muda* es una palabra japonesa para el trabajo o consumo de recursos que no añade valor desde la perspectiva del cliente. Una inspección necesaria para asegurar seguridad o calidad no desaparece por arte de magia: primero debemos distinguir lo indispensable de lo evitable.

En una sola mañana podríamos encontrar las siete formas clásicas. Hay **sobreproducción** al fabricar tapas que nadie pidió, **inventario** cuando ocupan un pasillo, **transporte** cuando el lote cruza dos veces la nave y **movimiento** cuando una persona tiene que agacharse por cada adaptador. Hay **espera** frente a la prueba, **sobreprocesamiento** al volver a capturar un número ya registrado y **defectos** cuando una tapa pierde agua. Después se popularizó un octavo desperdicio, el **talento desaprovechado**. La operadora que sabe por qué se tuerce la junta lleva tres meses enderezándola. ¿Cuándo le preguntamos qué cambiaría?[2]

Ocho pasos para resolver una fuga



Figura 10.4 — Identificar desperdicios no basta para corregirlos; primero hay que describir exactamente qué falla.

Identificar desperdicios no basta para corregirlos. Si seis botellas de cada cien tienen fugas, podemos culpar a una persona, ajustar la máquina sin datos o investigar antes de actuar. Toyota desarrolló una disciplina práctica de resolución de problemas que comienza por describir exactamente qué falla y termina comprobando si la solución se mantiene. Sus **ocho pasos** no son una ceremonia: impiden saltar de la molestia a una solución favorita sin entender la causa.

Los desperdicios ayudan a mirar. Los ocho pasos ayudan a intervenir. Así se verían en un ejercicio de taller con cifras ilustrativas.[3]

TABLA 10.1 — OCHO PASOS PARA RESOLVER UNA FUGA

PASO	QUÉ HARÍAMOS CON LA TAPA
1. Aclarar el problema	De cada 100 botellas probadas, seis presentan fuga. Identificar cómo se hizo la prueba y cuál era el criterio de aceptación.
2. Desglosarlo	Separar resultados por modelo, lote de junta, turno y puesto para localizar dónde se concentra la falla.
3. Fijar una meta	Reducir la fuga del modelo afectado a una por cada 100 unidades o menos en cuatro semanas.
4. Investigar la causa raíz	Observar el montaje y comprobar si la junta se desplaza al apretar la tapa. Reproducir la falla antes de atribuirla al operador.
5. Diseñar contramedidas	Comparar una guía de montaje con un cambio en el alojamiento de la junta. Elegir qué probar primero y por qué.
6. Aplicar la contramedida	Probar la guía en un puesto y registrar qué lotes y turnos la usan.
7. Evaluar proceso y resultado	Comparar tasas de fuga en condiciones equivalentes y revisar si aumentó el tiempo de ensamble.
8. Estandarizar y compartir	Si la mejora se sostiene, actualizar el método, capacitar a los otros turnos y verificar su desempeño.

A veces encontrar la causa exige algo más que observación. Dos lotes hechos con las mismas instrucciones pueden comportarse de forma diferente por pequeñas variaciones en materiales, mediciones o máquinas. **Six Sigma** es un enfoque de mejora de calidad desarrollado en Motorola durante la década de 1980, que utiliza datos y herramientas estadísticas para reducir defectos y variación. No sustituye a Lean: puede complementar la observación del flujo cuando necesitamos distinguir patrones reales del azar.

Una de sus rutas de trabajo se llama **DMAIC**, por las iniciales en inglés de *Define, Measure, Analyze, Improve, Control*: definir, medir, analizar, mejorar y controlar. Con DMAIC definimos qué cuenta como fuga, medimos con un instrumento confiable, analizamos la variación, probamos una mejora y dejamos un control que avise si el proceso vuelve a desviarse. Dos turnos pueden aplicar el mismo torque promedio y tener resultados distintos si uno tiene mucha más dispersión. Cambiar la especificación basándonos sólo en la media podría empeorar los dos.[4]



Figura 10.5 — La tolerancia que el diseñador anotó en un plano se convierte en una consecuencia íntima para quien usa la botella a diario. Vista conceptual.

Lean sigue vigente porque esa distancia entre la intención de la dirección y lo que puede cumplir el último puesto se vuelve a abrir con cada producto.

La mejora local no alcanza cuando las prioridades de diseño, compras, ventas y operación se contradicen: la fábrica puede hacer perfectamente aquello que la empresa decidió equivocadamente, como producir botellas sin prever la reposición de tapas. Para alinear esos niveles, organizaciones japonesas adoptaron **Hoshin Kanri**, una práctica de despliegue de objetivos que conecta las decisiones estratégicas con planes concretos y permite revisar compromisos entre áreas. Si ventas propone ofrecer tapas para diez modelos y compras sólo puede asegurar cuatro, esa conversación tiene que ocurrir antes de imprimir la promesa en la caja.[5]

Cómo llegó esta idea al software

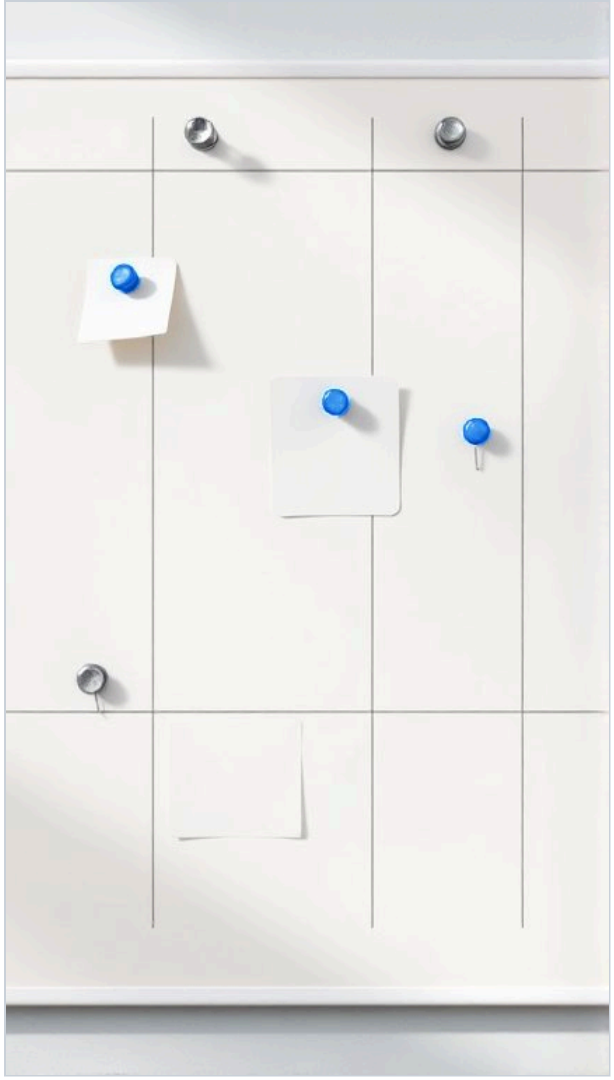


Figura 10.6 — Equipos que avanzan juntos, con actividades superpuestas en lugar de etapas sucesivas.

En la fábrica conocemos el diseño de la botella y queremos producir muchas unidades iguales. ¿Qué pasa cuando todavía estamos descubriendo cómo debe funcionar el producto? No podemos exigir que un equipo repita cien veces una solución que aún no ha encontrado.

En el desarrollo tradicional de software fue frecuente organizar el trabajo en etapas sucesivas: primero se definían todos los requisitos, después se diseñaba, luego se programaba y al final se probaba. Esa separación facilitaba planificar responsabilidades, pero podía hacer que los errores de interpretación aparecieran muy tarde. De esa dificultad surgieron prácticas que favorecían la colaboración temprana, las entregas parciales y la posibilidad de corregir la dirección a partir de resultados reales. Con el tiempo se las empezó a agrupar bajo la idea de **agilidad**: adaptarse a lo aprendido sin perder la disciplina de entregar.

En 1986, Hirotaka Takeuchi e Ikujiro Nonaka describieron equipos de desarrollo de productos que avanzaban juntos, con actividades superpuestas; la imagen que eligieron fue la de un equipo de rugby. Aquella observación industrial inspiró después a **Scrum**, un marco de trabajo en el que un equipo colabora durante periodos cortos, muestra un resultado concreto y ajusta su plan con lo aprendido. En 2001, los firmantes del **Manifiesto Ágil** articularon valores compartidos por varias corrientes del desarrollo de software, entre ellas Scrum, Extreme Programming y Crystal: colaboración, software funcionando, relación con el cliente y respuesta al cambio.[6, 7]

Lo que podemos trasladar de la fábrica es la capacidad de ver el trabajo acumulado, acortar el tiempo entre un error y su detección, y darle al equipo autoridad para corregir. Lo que cambia es la naturaleza de la unidad producida. En el taller queremos que la botella doscientas diez conserve las propiedades de la primera; en software suele bastar una sola versión del programa para atender a muchos compradores.

1986

TAKEUCHI Y NONAKA DESCRIBEN EL EQUIPO DE RUGBY

2001

SE FIRMA EL MANIFIESTO ÁGIL

MVP

PRODUCTO MÍNIMO VIABLE, NO UN PRODUCTO MALO

El producto mínimo viable



Pero entregar software con frecuencia no garantiza que alguien quiera usarlo. Podemos crear un buscador impecable de tapas que nadie compra porque el envío cuesta más que una botella nueva. **Lean Startup**, un enfoque popularizado por Eric Ries a partir de experiencias de emprendimiento y aprendizaje validado, responde a esa incertidumbre: antes de



Figura 10.7 — Antes de financiar un catálogo inteligente, imprimir el código en un grupo de empaques y surtir la tapa correcta a mano.

invertir mucho, formular una suposición comprobable, construir la prueba más pequeña que permita aprender y decidir con evidencia si continuamos o cambiamos. A esa prueba inicial se le suele llamar **producto mínimo viable** o *MVP*; no significa un producto malo, sino lo mínimo que permite comprobar una hipótesis con usuarios reales.[8]

Nuestra primera hipótesis podría ser que la gente compraría la refacción si supiera identificarla. Antes de financiar un catálogo inteligente, podemos imprimir el código del modelo en un grupo de empaques y ofrecer asistencia manual para surtir la tapa correcta. Contaríamos solicitudes, compras, errores y costo de atención. Si la mayoría identifica la pieza pero abandona al conocer el costo de envío, acabamos de cambiar la decisión sobre qué construir. Ese es el sentido de construir, medir y aprender.

El recorrido de un cambio



Para el diseñador de producto, el flujo suele ser visible hasta la entrega de las pantallas. Después aparece una tarjeta que dice «en desarrollo» y pasan varios días. El cambio que nos interesa es pequeño de enunciar: una persona escribe el código grabado en la base de su botella, ve la tapa compatible y puede pedirla. Para cumplirlo



Figura 10.8 — Abrir una rama: una línea de trabajo que cambia el buscador sin alterar todavía la versión compartida.

necesitamos que hablen entre sí la interfaz, el catálogo, el inventario y el sistema que recibe el pedido.

El **código** es el conjunto de instrucciones que ejecuta un programa. El **repositorio** es un lugar compartido donde se guarda ese código y su historial de cambios, como una bitácora que permite recuperar versiones anteriores. Y un sistema de **control de versiones**, como Git, registra quién propuso cada modificación y ayuda a combinar el trabajo de varias personas sin sobrescribirlo accidentalmente. No estamos ante una fábrica que envía más botellas: cada cambio modifica las instrucciones con las que atenderemos a todos los compradores.

Diseño descubre que el grabado es difícil de leer y agrega una ilustración que enseña dónde buscarlo. Ingeniería pregunta qué debe pasar si la persona confunde un cero con una O. La persona responsable de **aseguramiento de calidad**, o *QA* por *Quality Assurance*, propone probar una botella cuyo código existe aunque su tapa ya salió del catálogo. Estas preguntas modifican el diseño mientras todavía es barato mover una decisión.

TABLA 10.2 — LO QUE SE OYE EN EL EQUIPO

LO QUE SE OYE	QUÉ ACABA DE SUCEDER
«Abrí una rama»	En el repositorio se abre una línea de trabajo para cambiar el buscador sin alterar todavía la versión compartida.
«Hice un commit»	La desarrolladora registra localmente un cambio identificable y explica su intención.
«Ya hice push»	Envió sus commits al repositorio remoto. Sus colegas pueden verlos y las comprobaciones configuradas pueden ejecutarse.

LO QUE SE OYE	QUÉ ACABA DE SUCEDER
«Abrí un pull request»	Pide revisar e integrar la rama. El equipo ve las diferencias, los resultados de pruebas y las preguntas pendientes.
«Ya se hizo merge»	Los cambios quedaron integrados en la rama de destino.
«Está desplegado»	Una versión identificada se instaló en un entorno, por ejemplo producción.

¿Cada cuánto se hace *push*? Cuando hay un cambio coherente que compartir, una revisión que pedir o un punto de colaboración que conviene preservar. Como acuerdo inicial para un equipo que trabaja con integración continua, buscaría integrar cambios pequeños a la rama principal al menos diariamente, con pruebas en verde. Un *push* diario a una rama aislada durante tres semanas deja intacto el problema de integración.[9, 10]

El siguiente problema aparece cuando integrar una modificación puede romper otra que ya funcionaba. Para detectarlas antes, los equipos empezaron a automatizar la construcción y las pruebas del programa cada vez que incorporaban cambios. De ahí nace la práctica de **integración continua**.

CI/CD, DevOps y DevSecOps



La **integración continua (CI)** reúne los cambios frecuentemente y ejecuta comprobaciones automáticas. Cuando alguien modifica el catálogo, el sistema construye el programa y verifica, entre otras cosas, que los códigos anteriores sigan devolviendo su tapa. La **entrega continua (CD)** mantiene una versión verificada en condiciones de desplegarse bajo decisión del



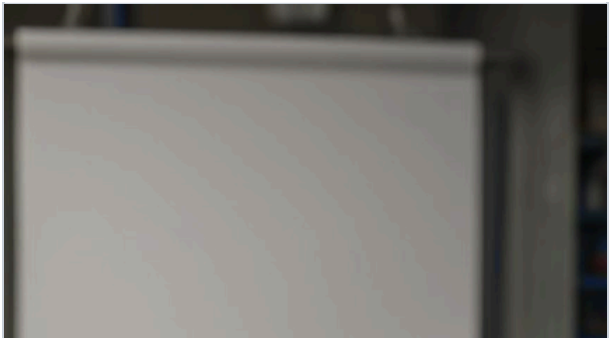
Figura 10.9 — Comprobaciones automáticas cada vez que un cambio llega al repositorio compartido.

equipo. En **despliegue continuo**, los cambios que superan los controles establecidos llegan automáticamente a producción.

Una aplicación no termina cuando alguien escribe el código. También hay que mantenerla disponible, proteger los datos y reaccionar ante fallas. A la colaboración sostenida entre desarrollo (*Development*) y operaciones (*Operations*) se le llamó **DevOps**; al hacer explícita la seguridad (*Security*) a lo largo del proceso se habla de **DevSecOps**.

Para consultar compatibilidad basta el código de la botella. Pedirle al comprador su correo en esa pantalla añadiría datos que custodiar sin ayudarlo a encontrar la tapa. Esa es una discusión de seguridad en diseño. En el repositorio se revisan permisos y dependencias; en producción se monitorean errores y se sabe quién puede desactivar el buscador si empieza a cruzar modelos.[11]

Qué sigue quien coordina la entrega



Aquí aparece una responsabilidad distinta de diseñar pantallas o programar funciones: conseguir que el trabajo de varias personas llegue a convertirse en una capacidad que el cliente pueda utilizar. A menudo se llama **Project**



Figura 10.10 — Aceptar surtir los primeros pedidos a mano si operación puede hacerlo y el cliente conoce el plazo.

Manager (PM) a quien coordina alcance, fechas, presupuesto, riesgos y dependencias de un proyecto; cuando se habla de *delivery* se enfatiza la entrega efectiva, no sólo el cumplimiento de actividades.

Una manera costosa de organizar este producto sería entregar primero las seis pantallas, después todos los servicios y al final conectar inventario. Podrían pasar semanas con tres equipos reportando avance sin haber vendido una sola tapa. Elegir una familia de botellas y llevar su recorrido completo hasta que alguien reciba la refacción correcta —aceptando surtir los primeros pedidos a mano si operación puede hacerlo— es **agregar valor de forma iterativa**. El primer incremento recupera el uso de una botella; el segundo evita cobrar por una tapa agotada.

En la revisión de entrega conviene ver un pedido de punta a punta: cuántos días lleva el cambio más viejo sin integrarse, cuánto tiempo espera una revisión, qué dependencia impide probar con inventario y qué defectos han vuelto a abrirse.

Un tablero con cinco tarjetas detenidas frente a una sola persona que puede

aprobar el catálogo se parece bastante a la fila de botellas esperando el adaptador.

Abrir otras cinco historias ocuparía al equipo. Conseguir ese adaptador cambiaría la fecha de entrega. El cuello de botella rara vez está en cuántas tareas abrimos; está en quién puede destrabarlas.

El defecto que sobrevive al demo

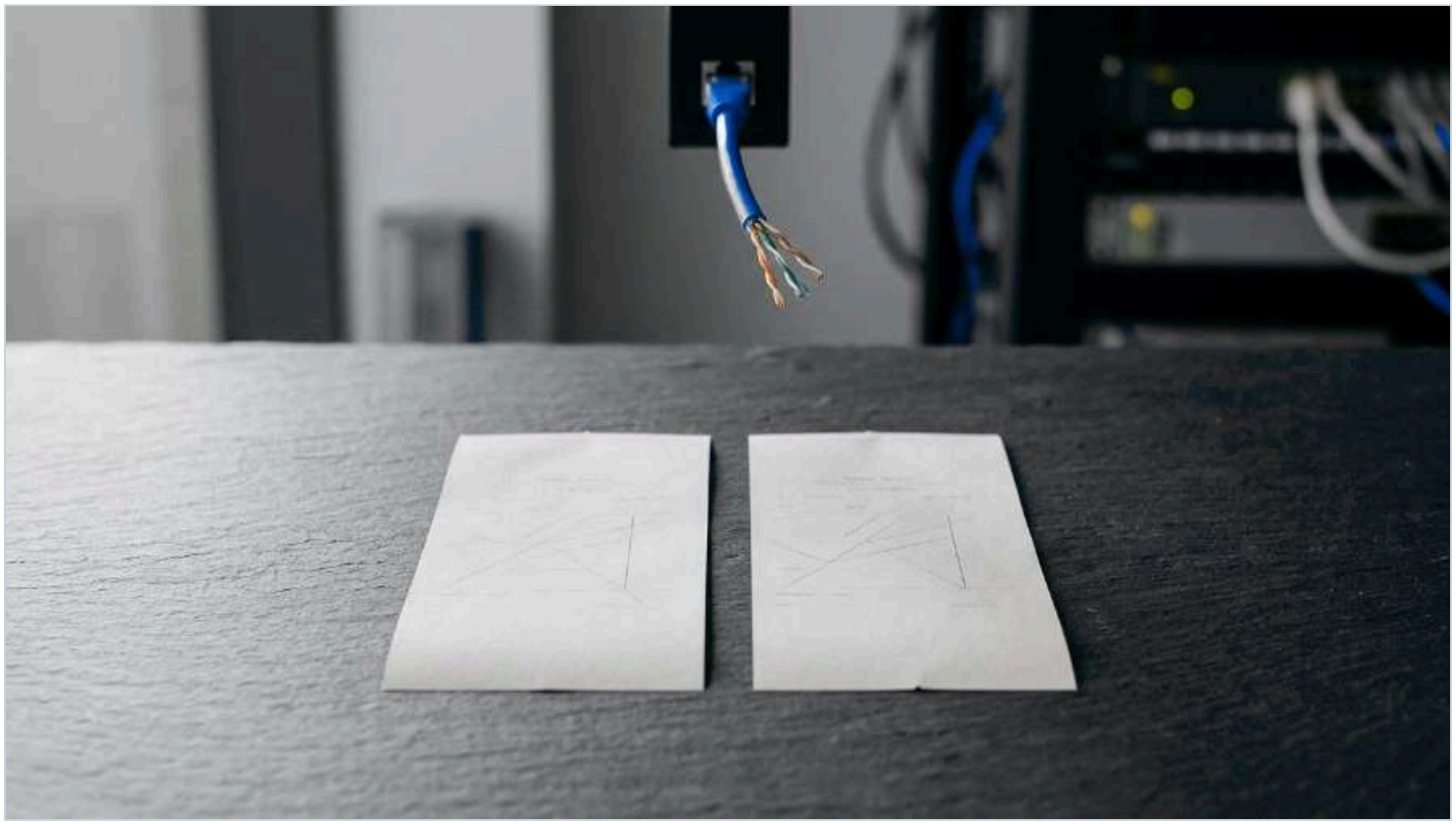


FIGURA 10.11 QA corta la conexión justo después de confirmar. El cliente reintenta y aparecen dos pedidos.

En la demostración escribimos un código, elegimos una tapa y vemos el pedido confirmado. QA hace algo menos amable y mucho más parecido a la vida real: corta la conexión justo después de confirmar. El cliente reintenta y aparecen dos pedidos. La pantalla se ve exactamente como la aprobó diseño. El cliente acaba de pagar dos veces.

El ciclo de corrección empieza al registrar versión, entorno, pasos, resultado esperado y evidencia de los dos cobros. En una revisión inicial de defectos llamada *triage* —término tomado de la clasificación por urgencia—, calidad, desarrollo y producto valoran impacto, alcance y prioridad. Desarrollo reproduce el fallo, corrige el manejo de reintentos y añade una prueba que intente duplicar el pedido. QA repite el caso en la nueva versión y comprueba por regresión que una segunda compra legítima siga siendo posible. Si ya hubo cobros reales, operación necesita además identificar a los afectados y resolverlos: corregir código y devolver dinero son dos trabajos distintos.

Hasta ahora hemos preguntado si el programa funciona técnicamente. Falta saber si sirve para realizar el trabajo real. **UAT** corresponde a *User Acceptance Testing*, o prueba de aceptación por parte de usuarios de negocio: personas que comprueban si pueden completar situaciones representativas con el resultado esperado. En muchos equipos interviene un **Product Owner**, la persona que prioriza necesidades del producto y acuerda con negocio qué resultados se consideran aceptables. Si la tienda promete tres días y almacén necesita ocho, las pruebas técnicas pueden estar en verde y el producto seguir sin estar listo para anunciarse.

Figura 10.12



Corregir código y devolver dinero son dos trabajos distintos, aunque empiecen en el mismo defecto.

Escuchar al cliente y medir el efecto



No todas las preguntas se contestan con una prueba informática. Para comprender por qué una persona elige mal una tapa podemos empezar escuchándola. Un **grupo de enfoque**, o *focus group*, reúne a participantes para conversar y observar reacciones ante una idea, objeto o experiencia. Sirve para descubrir palabras, motivaciones y confusiones; por sí solo no permite afirmar qué hará toda la población.

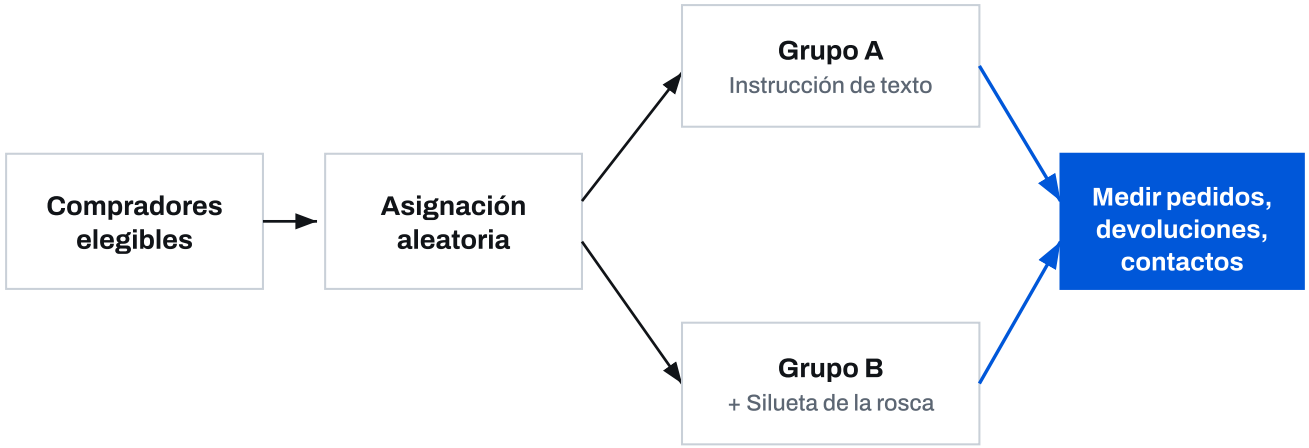
Pongamos una botella y dos empaques sobre la mesa. Una persona gira la botella para buscar el modelo en la base; otra lee «serie» en la caja y copia el número equivocado; alguien propone imprimir una silueta de la rosca. Que cuatro participantes prefieran la silueta todavía deja abierta la pregunta sobre cuántas compras correctas produciría.

Figura 10.13 — Pidamos a los participantes que encuentren la refacción y expliquen qué están mirando.

En digital podemos observar sesiones de uso con la misma cercanía y después probar una hipótesis a escala. Una **prueba A/B** es un experimento comparativo: dos grupos semejantes reciben versiones distintas y medimos cuál produce mejor el resultado que nos importa. La asignación aleatoria busca evitar que una versión parezca superior sólo porque la usaron personas diferentes. A muestra la instrucción de texto y B añade la silueta; acordamos de antemano medir pedidos compatibles por visitante asignado y vigilar devoluciones, cancelaciones y contactos a soporte.

Si el tráfico apenas alcanza para vender diez tapas por semana, una comparación estadística puede tardar más de lo que vale la decisión. Preferiría entonces observar compradores, acompañar pedidos y reconocer cuánto queda por aprender.[12]

Figura 10.14



Efecto mínimo, tamaño de muestra y ventana de observación se fijan antes de mirar resultados.

Mirar el primer día y declarar ganadora a la versión con más clics puede premiar la ilustración que más confunde.

Cuánto cuesta llegar al mercado

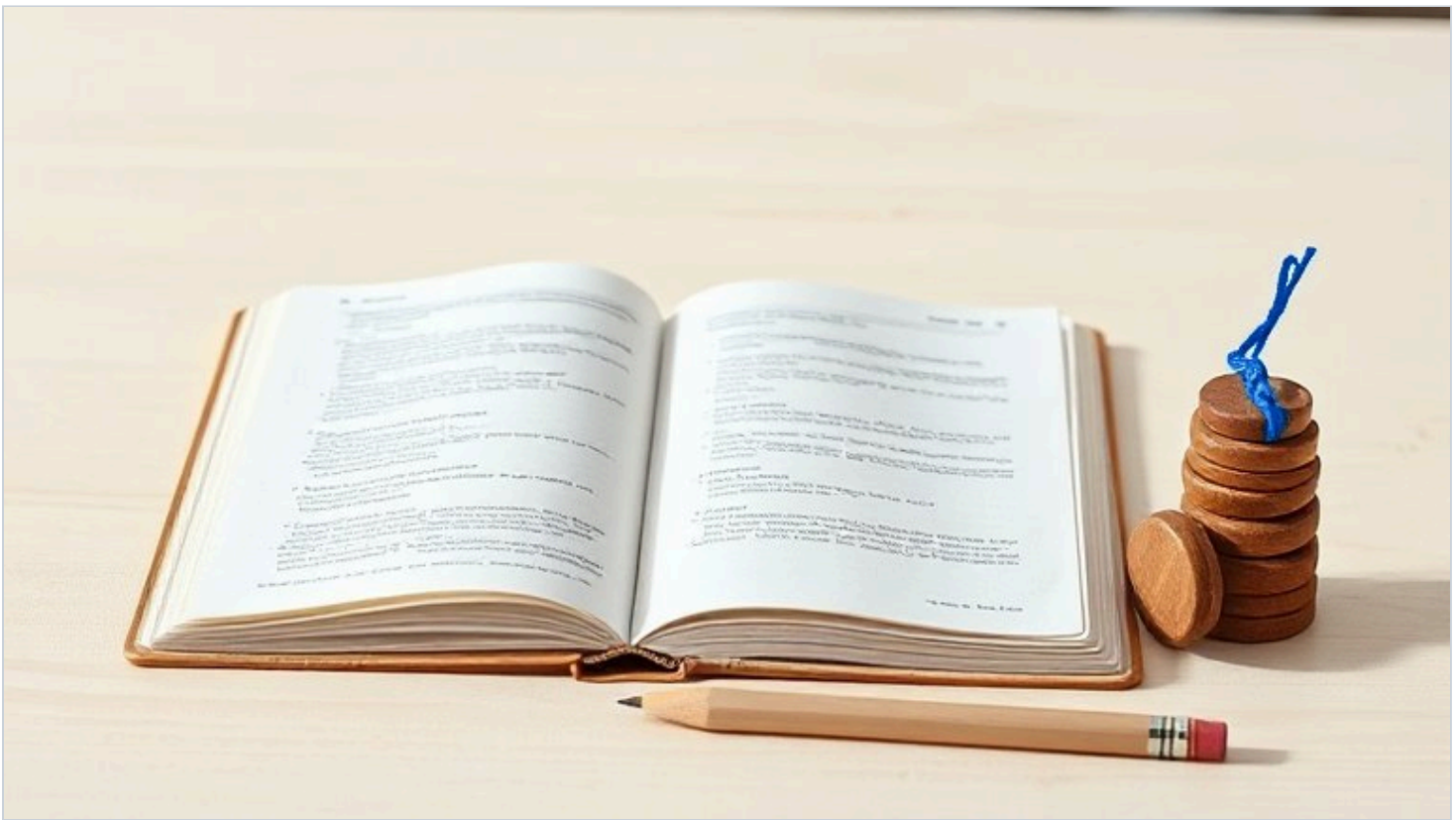


FIGURA 10.15 Apartar una proporción para lanzamiento y primeras iteraciones, y autorizar el resto antes de abrir al público.

La fábrica conoce la diferencia entre aprobar una muestra y autorizar un lote. En digital también necesitamos distinguir la versión que funciona de la decisión de ponerla en manos de miles de personas. La siguiente conversación tendría que ocurrir antes de comprar publicidad, cuando todavía podemos modificar la promesa.

TABLA 10.3 — DECISIÓN DE PRELANZAMIENTO

DECISIÓN DE PRELANZAMIENTO	BOTELLA Y REFACCIÓN	SERVICIO DIGITAL
Qué se promete y a quién	Modelos compatibles, precio, garantía, canales de distribución	Segmentos habilitados, stock por región, precio total y plazo visibles
Qué demuestra que se puede entregar	Lote piloto, hermeticidad y ensamble verificados	Pedido completo probado, UAT, monitoreo y recuperación ensayada
Cuánto volumen se puede atender	Capacidad, existencias de seguridad y reposición	Capacidad de la plataforma, inventario sincronizado y personas para surtir y atender
Qué se va a comunicar	Empaque, instructivo, materiales para tienda y distribuidores	Página de búsqueda, correo, campaña, guion de soporte y fecha de activación
Quién decide detener	Responsable de calidad inmoviliza el lote y rastrea unidades	Responsable de guardia desactiva la función, conserva trazabilidad y coordina atención

Construir y lanzar también son decisiones económicas. **Presupuesto** es el dinero autorizado para un propósito; **gasto ejercido**, lo que ya se utilizó; **compromisos**, obligaciones asumidas aunque todavía no se hayan pagado; y **pronóstico**, una estimación de cuánto acabaremos gastando. Confundir esas cantidades puede hacer parecer sano un proyecto que ya prometió más recursos de los que tiene.

Pondría presupuesto a esas decisiones. Supongamos, para este ejercicio, un techo aprobado de **un millón de pesos mexicanos**. Apartamos **400 mil** para lanzamiento y primeras iteraciones, y autorizamos hasta **600 mil** antes de abrir al público. En una línea que requiere moldes nuevos el perfil sería otro; en ambos casos la cantidad reservada tiene que cubrir obligaciones concretas: distribución, medios, soporte, medición y tiempo para corregir una primera versión.

Si ya se ejercieron 480 mil, hay órdenes comprometidas pendientes de pago por 70 mil y calculamos otros 90 mil de trabajo todavía sin contratar, el costo previsto antes del lanzamiento es de **640 mil**. Aunque tesorería sólo haya pagado 480 mil, ya proyectamos rebasar el límite por **40 mil**. El PM debe presentar la decisión ahora: podría aplazar una variante de color si eso libera los 40 mil sin perjudicar la promesa del piloto, pedir una ampliación respaldada por evidencia o suspender la salida si lo pendiente compromete la confiabilidad.

Tampoco basta con pronosticar ventas. Si de cada pedido quedan 80 pesos después de fabricación, envío subsidiado, comisiones, devoluciones esperadas y atención variable, pagar 100 pesos por adquirir una sola compra deja un faltante de 20. Se puede decidir financiarlo si hay evidencia de compras posteriores. En una refacción diseñada para durar años, pediría ver esa evidencia antes de firmar la pauta.

$1M	$640K	$40K
TECHO APROBADO PARA EL EJEMPLO	COSTO PREVISTO ANTES DEL LANZAMIENTO	SOBRE EL LÍMITE, AÚN SIN GASTARSE

Las dos curvas de trabajo

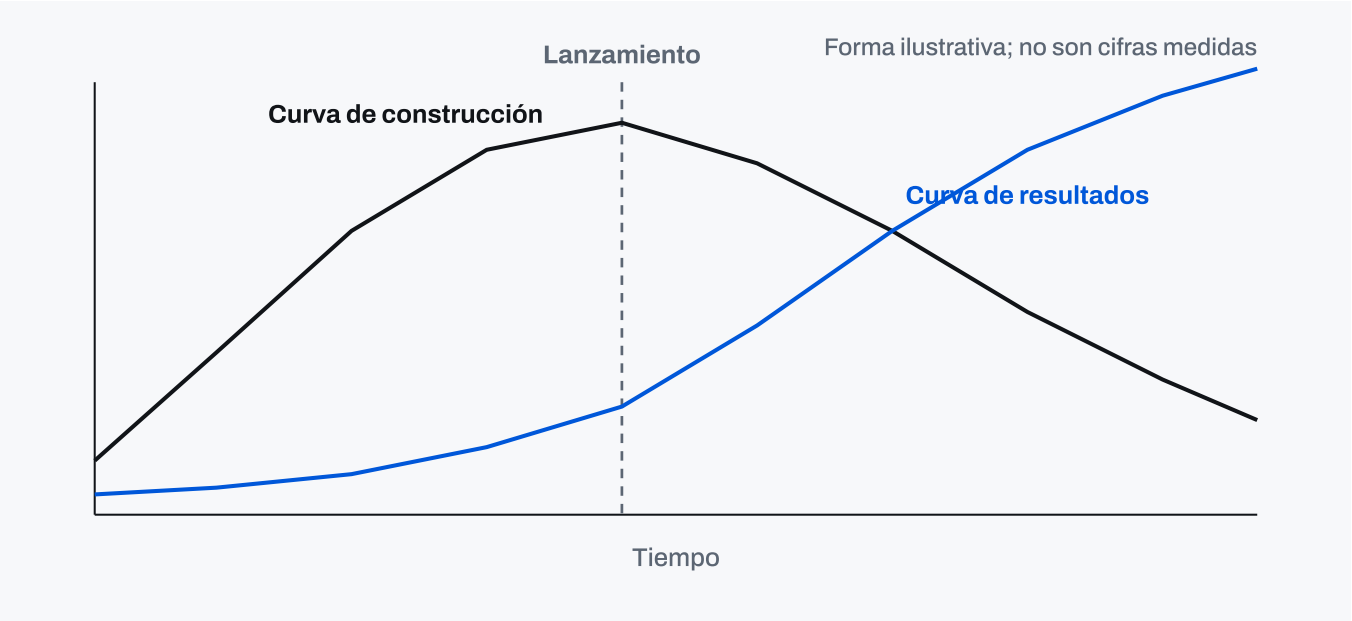


Figura 10.16 — El diseñador puede generar veinte composiciones de una tapa en una mañana; su responsabilidad aumenta al elegir cuál es correcta.

Durante años fue fácil defender el presupuesto de construcción con algo que se podía enseñar: un molde, una máquina, una pantalla terminada. El equipo se hacía grande para llegar al lanzamiento. A la mañana siguiente empezaban las preguntas sobre venta, adopción y permanencia, a veces con menos gente y con el presupuesto ya consumido. La **inteligencia artificial (IA)** puede reducir parte del esfuerzo de producir materiales y programas; no elimina el trabajo de averiguar si lo producido es correcto, útil o sostenible. Por eso dibujo ese proyecto con dos curvas de esfuerzo.

La primera curva empieza antes de escribir código. Crece con la observación, las decisiones de diseño, la arquitectura y las pruebas. Un agente puede proponer el buscador de tapas y generar casos de prueba en la misma tarde; alguien todavía tiene que descubrir que dos roscas comparten nombre, conseguir el catálogo correcto y decidir cómo tratar un modelo descontinuado. Si esa decisión llega tarde, la herramienta también puede ayudarnos a implementar más rápido la respuesta equivocada.[13]

Figura 10.17



Si el pico de resultados llega el mismo día que el pico de construcción, retiramos recursos justo cuando por fin podemos aprender de compradores reales.

La segunda curva arranca antes de abrir al público, cuando preparamos distribución, comunicación e instrumentación; puede crecer después del lanzamiento y superar a la primera. Ahora hay que averiguar por qué una persona vio el anuncio, encontró el modelo y abandonó al conocer el envío. El resultado tiene su propio calendario: las impresiones llegan enseguida, las compras después y las devoluciones semanas más tarde.

El diseñador puede generar veinte composiciones de una tapa en una mañana; su responsabilidad aumenta cuando una de esas imágenes representa una rosca que almacén jamás podría surtir. Quien trabaja en **growth**, es decir, en mejorar de forma medible la adquisición, activación y permanencia de clientes, puede preparar más experimentos con el mismo equipo, aunque su tráfico disponible siga alcanzando para resolver una sola pregunta. Quien trabaja en **SEO** (*Search Engine Optimization*, u optimización para buscadores) puede ordenar las dudas que llegan a soporte y convertirlas en una guía que

ayude a identificar un modelo antiguo; generar cientos de páginas casi iguales ocupa poco tiempo y sigue sin contestar esa pregunta.[14]

Quiero que las primeras iteraciones posteriores a producción estén planteadas desde el diseño. El día que abramos el piloto, alguien debe conocer el costo por pedido correcto, alguien debe poder llamar al cliente que devolvió una tapa y alguien debe tener autoridad para cambiar el anuncio o detener la venta. El dinero reservado después del lanzamiento compra esa capacidad de actuar mientras el problema todavía tiene nombre y apellido.



Figura 10.18 — A la operadora que lleva tres meses enderezando juntas le preguntaría qué cambiaría del molde.

Mi lectura del mercado digital es incómoda: vamos a necesitar más gente que piense y cuestione, y menos puestos dedicados exclusivamente a ejecutar instrucciones. A la operadora que lleva tres meses enderezando juntas le preguntaría qué cambiaría del molde. A la diseñadora que puede generar veinte anuncios le preguntaría cuál pondría frente al comprador y qué resultado la haría retirarlo.

Las dos conocen algo que el director difícilmente verá desde su escritorio. Si vamos a invertir en inteligencia, empezaría por darles autoridad para usar la suya.

REFERENCIAS Y NOTAS

La fábrica de botellas térmicas y sus cifras son un ejercicio didáctico construido para este capítulo, no el registro de un proyecto real. Las referencias externas se consultaron el 9 de noviembre de 2026.

1. Toyota Motor Corporation, Toyota Production System. Principios de just-in-time y jidoka. ↩
2. Lean Enterprise Institute, The Eight Wastes of Lean. ↩
3. Lean Enterprise Academy, Back to Basics: Problem Solving at Toyota. Los ocho pasos de resolución de problemas. ↩
4. American Society for Quality, DMAIC Process. ↩
5. Lean Enterprise Institute, Hoshin Kanri. ↩
6. Hirotaka Takeuchi e Ikujiro Nonaka, The New New Product Development Game, Harvard Business Review, 1986. ↩
7. Jim Highsmith, History of the Agile Manifesto, 2001. ↩
8. Eric Ries, The Lean Startup Methodology. ↩
9. GitHub Docs, GitHub flow. ↩
10. DORA, Trunk-based development. ↩
11. NIST, Secure Software Development Framework, SP 800-218. ↩
12. Microsoft Research, Patterns of Trustworthy Experimentation. ↩
13. DORA, State of AI-assisted Software Development, 2025. ↩
14. Google Search Central, Guía sobre el uso de IA generativa en sitios web. ↩

Historial de edición — Capítulo 10

Versión	Fecha	Nota
v0.1	11 nov 2026	Primera edición bilingüe (ES/EN): de la fábrica al código, takt time y Lean, los ocho pasos, Hoshin Kanri, cómo llegó la idea al software, MVP, Git y CI/CD, DevOps, coordinar la entrega, el defecto que sobrevive al demo, escuchar al cliente y A/B, cuánto cuesta llegar al mercado, las dos curvas de trabajo.

← CAPÍTULO ANTERIOR

09 — La excelencia en ingeniería como decisión de producto

SIGUIENTE CAPÍTULO →

11 — en desarrollo

© 2026 J. Angel Sosa. *Pensar en Productos, Construir Sistemas*.